

DEPARTMENT OF INFORMATION TECHNOLOGY

24ITT32 - PROGRAMMING IN JAVA

INDIVIDUAL PROJECT ON
ACADEMIC MANAGEMENT SYSTEM

P R O J E C T R E P O R T

Submitted by

THANYA P

25ITR157

JAVA ASSIGNMENT

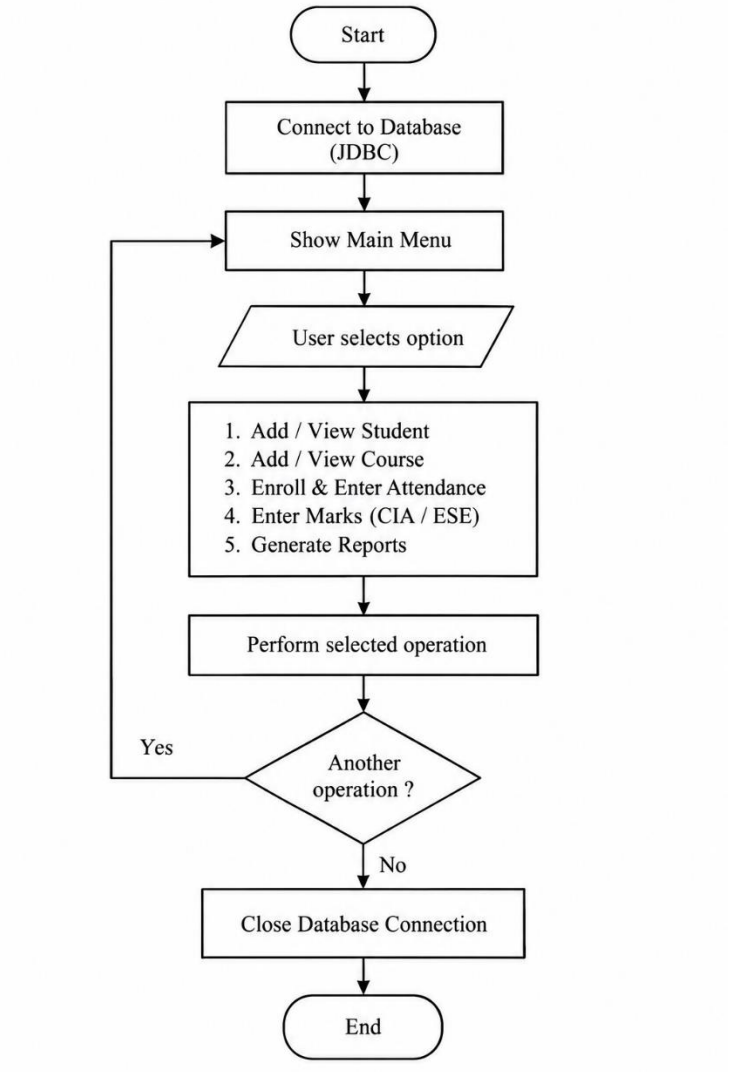
AIM:

Development of Kongu Engineering College (R2024) Student Academic & Continuous Evaluation System using Java JDBC.

OBJECTIVE:

Design and develop a database-driven Java application using JDBC to manage student records, perform incremental marks entry (Continuous Internal Assessments and End Semester Examinations), enforce attendance eligibility rules, and generate formal academic performance reports according to Kongu Engineering College (Autonomous) - Regulation 2024. System Requirements & Specifications.

FLOW CHART:



PROCEDURE:

- Load the MySQL Connector/J driver to enable communication between the Java application and the MySQL database.

- Establish a connection to the kec_eval database running on the local MySQL server using DriverManager.getConnection().

- Store and retrieve student details, including roll number, name, department, and semester, along with course information such as course code, title, credits, and course type.

- Enroll students in semester courses, record the number of classes held and attended, and calculate the attendance percentage.

- Verify examination eligibility based on attendance percentage. Students with 75% or above are eligible, those with 65%–74% require medical condonation verification, and those below 65% are marked as Shortage of Attendance (SA).

- Record CIA-1, CIA-2, and assignment marks, and calculate the scaled internal assessment marks according to the specified theory and practical course weightages.

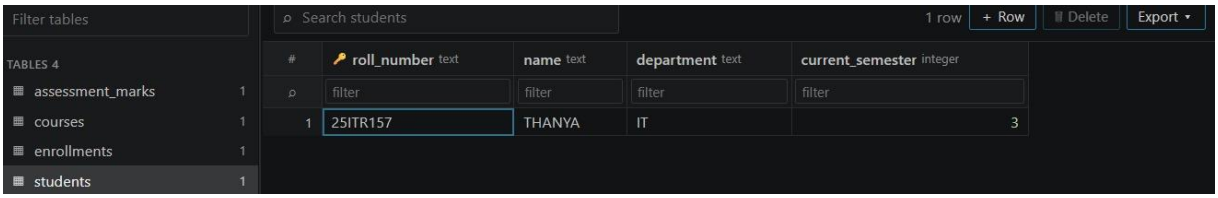
- Record end-semester examination marks out of 100, verify the minimum passing requirements, and scale the marks according to the applicable theory and practical .

- Calculate the total marks by combining scaled CIA and ESE marks. Assign letter grades and grade points according to the specified grading scale, marking students as RA if they fail the examination or score below the required passing mark.

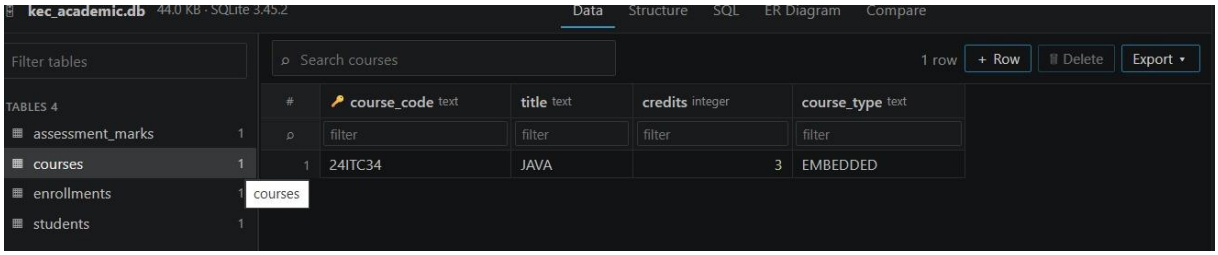
- Generate student grade sheets, attendance shortage reports, and department-wise pass percentage reports. Display the results through the interactive web dashboard and the command prompt console**.**

DATABASE SCHEMA:

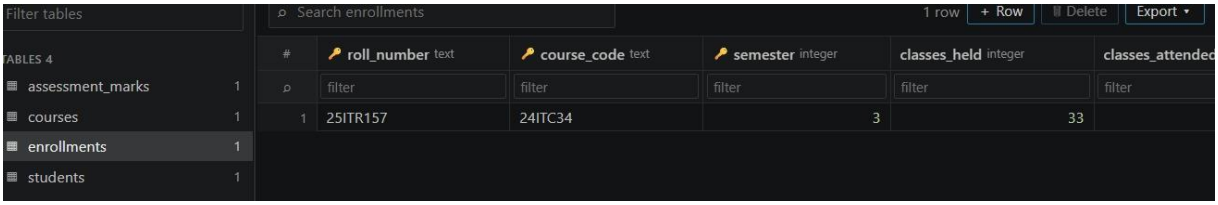
1.STUDENTS TABLE:



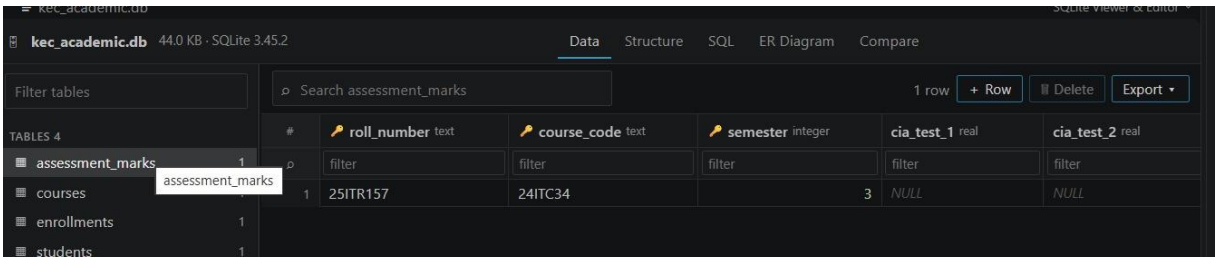
2. COURSES TABLE:



3. ENROLLMENTS TABLE:



4. ASSESSMENT MARKS TABLE:



SOURCE CODE:

MAINAPP.JAVA:

```
package edu.kec;

import java.sql.SQLException;
import java.util.Locale;
import java.util.Scanner;

public class MainApp {
    private final Scanner scanner = new Scanner(System.in);
    private final StudentDAO studentDAO = new StudentDAO();
    private final AcademicDAO academicDAO = new AcademicDAO();

    public static void main(String[] args) {
        try {
            DBConnection.getConnection().close();
            new MainApp().run();
        } catch (Exception exception) {
            System.err.println("Application error: " + exception.getMessage());
            exception.printStackTrace(System.err);
            System.exit(1);
        }
    }

    private void run() {
        System.out.println("KEC Academic Records - Regulation 2024");
        while (true) {
            System.out.println("""

                    1. Add student
                    2. Add course
                    3. Enroll student / update attendance
                    4. Enter CIA marks
                    5. Enter ESE mark / mark absent
                    6. Print student semester mark sheet
                    7. Print attendance defaulters (<65%)
                    8. Print department class summary
                    0. Exit
                    """);
```

```
            try{
             switch (readInt("Choice: ")) {
                    case 1 -> addStudent();
                    case 2 -> addCourse();
                    case 3 -> enrollAndUpdateAttendance();
                    case 4 -> enterCiaMarks();
                    case 5 -> enterEseMark();
                    case 6 -> printMarkSheet();
                    case 7 -> System.out.println(academicDAO.attendanceDefaulters(readSemester()));
                    case 8 -> System.out.println(academicDAO.classSummary(readSemester()));
                    case 0 -> {
                       System.out.println("Goodbye.");
                       return;
                    }
                    default -> System.out.println("Choose a listed menu option.");
                }
            } catch (SQLException | IllegalArgumentException | IllegalStateException exception) {
                System.err.println("Operation failed: " + exception.getMessage());
            }
        }
    }

    private void addStudent() throws SQLException {
        studentDAO.addStudent(readText("Roll number: "), readText("Student name: "),
                readText("Department: "), readSemester());
        System.out.println("Student added.");
    }

    private void addCourse() throws SQLException {
        String type = readText("Course type (THEORY/PRACTICAL/EMBEDDED):
").toUpperCase(Locale.ROOT);
        studentDAO.addCourse(readText("Course code: "), readText("Course title: "),
                readInt("Credits: "), type);
        System.out.println("Course added.");
    }

    private void enrollAndUpdateAttendance() throws SQLException {
        String roll = readText("Roll number: ");
        String course = readText("Course code: ");
        int semester = readSemester();
        academicDAO.enroll(roll, course, semester);
        System.out.println("Enrollment added. Record the current cumulative attendance.");
        updateAttendance(roll, course, semester);
    }

    private void updateAttendance(String roll, String course, int semester) throws SQLException {
        int held = readInt("Total classes held: ");
        int attended = readInt("Classes attended: ");
        boolean approved = readYesNo("Medical condonation approved? (y/n): ");
        academicDAO.updateAttendance(roll, course, semester, held, attended, approved);
        AcademicDAO.AttendanceStatus status = academicDAO.attendanceStatus(roll, course, semester);
```

```
        System.out.printf(Locale.ROOT, "Attendance: %.2f%% - %s%n", status.percentage(), status.message());
    }

    private void enterCiaMarks() throws SQLException {
        String roll = readText("Roll number: ");
        String course = readText("Course code: ");
        int semester = readSemester();
        boolean enterTest1 = readYesNo("Enter/update CIA Test 1? (y/n): ");
        if (enterTest1) {
            academicDAO.recordCiaMark(roll, course, semester, "cia_test_1", readMark("CIA Test 1 (0-100): "));
        }
        boolean enterTest2 = readYesNo("Enter/update CIA Test 2? (y/n): ");
        if (enterTest2) {
            academicDAO.recordCiaMark(roll, course, semester, "cia_test_2", readMark("CIA Test 2 (0-100): "));
        }
        boolean enterAssignment = readYesNo("Enter/update assignment mark? (y/n): ");
        if (enterAssignment) {
            academicDAO.recordCiaMark(roll, course, semester, "assignment_mark",
                    readMark("Assignment (0-100): "));
        }
        boolean enterQuiz = readYesNo("Enter/update quiz mark? (y/n): ");
        if (enterQuiz) {
            academicDAO.recordCiaMark(roll, course, semester, "quiz_mark", readMark("Quiz (0-100): "));
        }
        boolean enterLab = readYesNo("Enter/update lab evaluation mark? (y/n): ");
        if (enterLab) {
            academicDAO.recordCiaMark(roll, course, semester, "lab_evaluation_mark",
                    readMark("Lab evaluation (0-100): "));
        }
        System.out.println("CIA marks saved; omitted components were left unchanged.");
    }

    private void enterEseMark() throws SQLException {
        String roll = readText("Roll number: ");
        String course = readText("Course code: ");
        int semester = readSemester();
        AcademicDAO.AttendanceStatus status = academicDAO.attendanceStatus(roll, course, semester);
        System.out.printf(Locale.ROOT, "Attendance: %.2f%% - %s%n", status.percentage(), status.message());
        if (!status.eligibleForEse()) {
            System.out.println("ESE entry blocked by the attendance rule.");
            return;
        }
        boolean absent = readYesNo("Student absent in ESE? (y/n): ");
        Double mark = absent ? null : readMark("ESE mark (0-100): ");
        academicDAO.recordEseMark(roll, course, semester, mark, absent);
        System.out.println(absent ? "Absence recorded as AB." : "ESE mark saved.");
    }

    private void printMarkSheet() throws SQLException {
        System.out.println(academicDAO.markSheet(readText("Roll number: "), readSemester()));
    }
```

```
private int readSemester() {
    return readInt("Semester (1-8): ");
}

private double readMark(String prompt) {
    while (true) {
        double mark = readDouble(prompt);
        if (mark >= 0 && mark <= 100) {
            return mark;
        }
        System.out.println("Enter a mark from 0 through 100.");
    }
}

private int readInt(String prompt) {
    while (true) {
        try {
            return Integer.parseInt(readText(prompt));
        } catch (NumberFormatException exception) {
            System.out.println("Enter a whole number.");
        }
    }
}

private double readDouble(String prompt) {
    while (true) {
        try {
            return Double.parseDouble(readText(prompt));
        } catch (NumberFormatException exception) {
            System.out.println("Enter a number.");
        }
    }
}

private boolean readYesNo(String prompt) {
    while (true) {
        String response = readText(prompt);
        if (response.equalsIgnoreCase("y") || response.equalsIgnoreCase("yes")) return true;
        if (response.equalsIgnoreCase("n") || response.equalsIgnoreCase("no")) return false;
        System.out.println("Enter y or n.");
    }
}

private String readText(String prompt) {
    while (true) {
        System.out.print(prompt);
        String value = scanner.nextLine().trim();
        if (!value.isEmpty()) {
            return value;
        }
        System.out.println("A value is required.");}}}
```

DB CONNECTION.JAVA:

```
package edu.kec;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.nio.file.Files;
import java.nio.file.Path;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.stream.Collectors;

public final class DBConnection {
    private static final String SCHEMA_RESOURCE = "/kec_schema.sql";
    private static volatile boolean initialized;

    private DBConnection() {
    }

    public static Connection getConnection() throws SQLException {
        initialize();
        Connection connection = DriverManager.getConnection(databaseUrl());
        try (Statement statement = connection.createStatement()) {
            statement.execute("PRAGMA foreign_keys = ON");
        } catch (SQLException exception) {
            connection.close();
            throw exception;
        }
        return connection;
    }

    private static void initialize() throws SQLException {
        if (initialized) {
            return;
        }
        synchronized (DBConnection.class) {
            if (initialized) {
                return;
            }
            String configuredPath = System.getenv("KEC_DB_PATH");
            Path databasePath = Path.of(configuredPath == null || configuredPath.isBlank()
                    ? "kec_academic.db" : configuredPath).toAbsolutePath();
            Path parent = databasePath.getParent();
            try {
                if (parent != null) {
                    Files.createDirectories(parent);
                }
                Class.forName("org.sqlite.JDBC");
                try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + databasePath);
                     Statement statement = connection.createStatement()) {
                    statement.execute("PRAGMA foreign_keys = ON");
                    for (String sql : loadSchema().split(";")) {
                        if (!sql.isBlank()) {
                            statement.execute(sql);
                        }
                    }
                }
                initialized = true;
            } catch (IOException | ClassNotFoundException exception) {
                throw new SQLException("Could not initialize the SQLite database.", exception);
            }
```

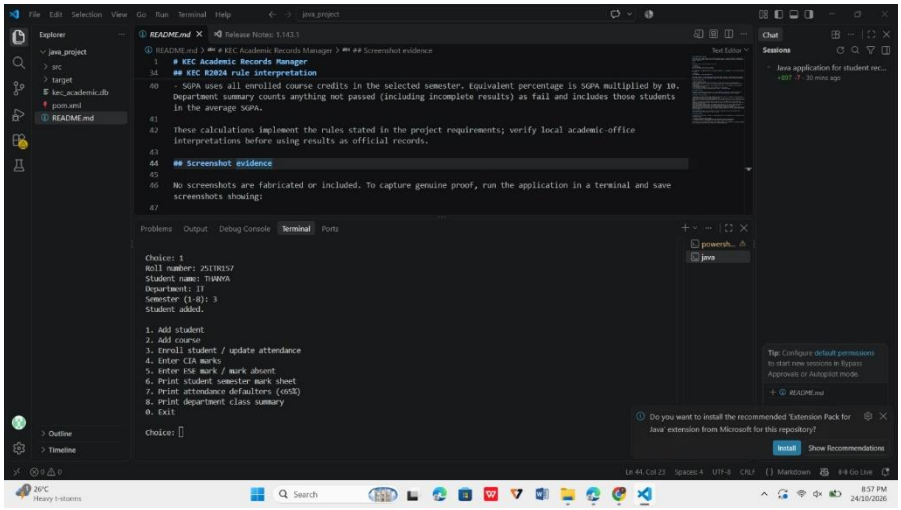
```
        }
    }

    private static String databaseUrl() {
        String configuredPath = System.getenv("KEC_DB_PATH");
        Path databasePath = Path.of(configuredPath == null || configuredPath.isBlank()
                ? "kec_academic.db" : configuredPath).toAbsolutePath();
        return "jdbc:sqlite:" + databasePath;
    }

    private static String loadSchema() throws IOException {
        try (InputStream input = DBConnection.class.getResourceAsStream(SCHEMA_RESOURCE)) {
            if (input == null) {
                throw new IOException("Schema resource not found: " + SCHEMA_RESOURCE);
            }
            try (BufferedReader reader = new BufferedReader(new InputStreamReader(input))) {
                return reader.lines().collect(Collectors.joining(System.lineSeparator()));
            }
        }
    }
}
```

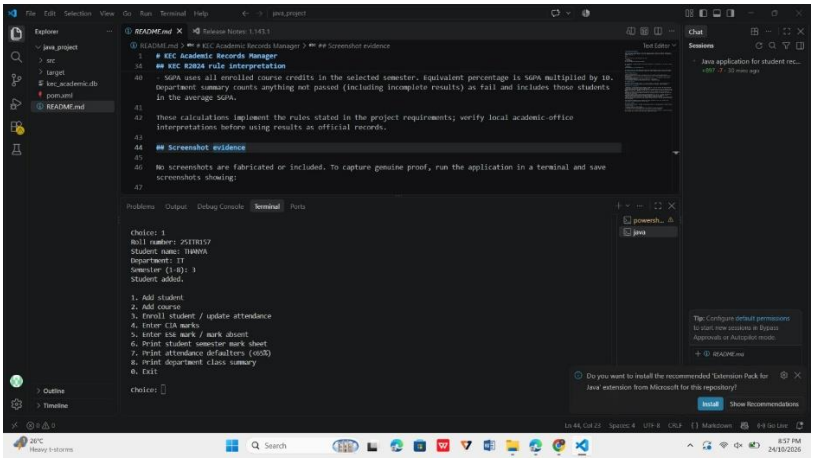
OUTPUT SCREENSHOTS:

MAIN MENU:

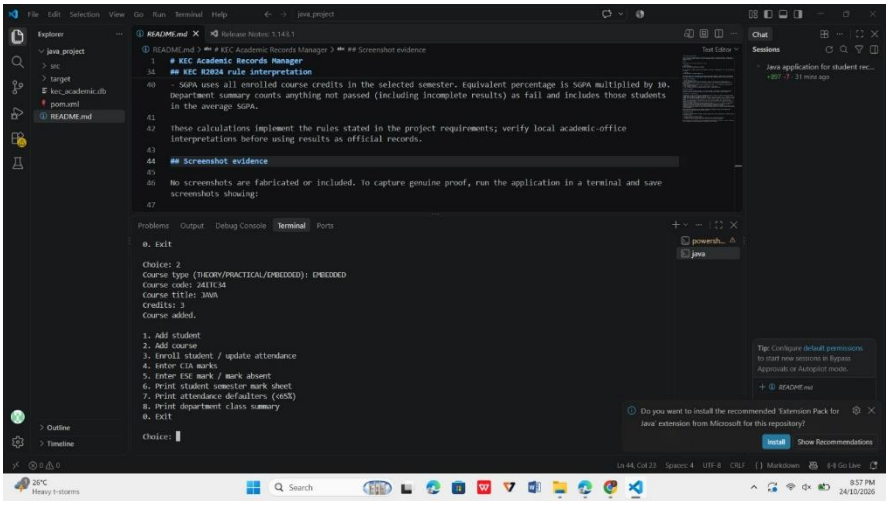


STUDENT & COURSE MANAGEMENT:

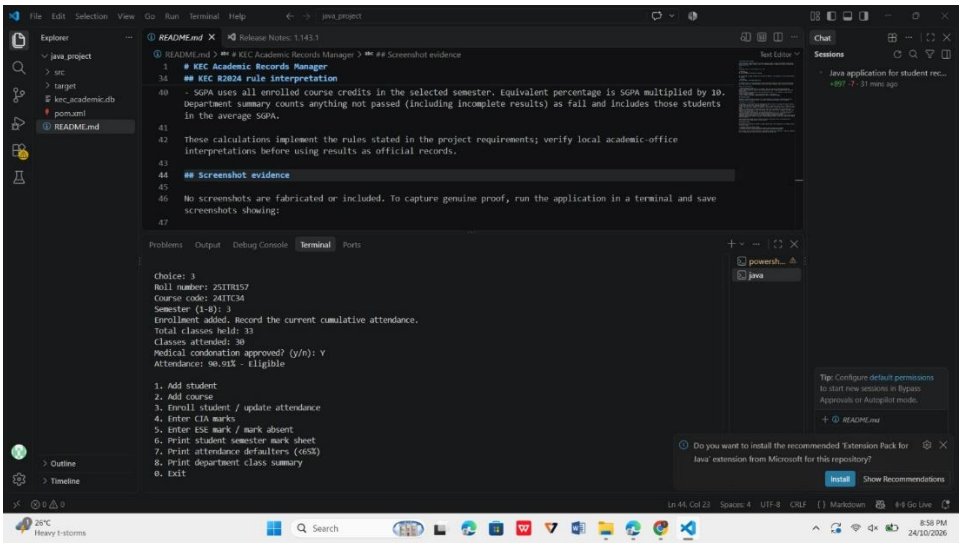
1.STUDENT REGISTRATION:



2. COURSE CATALOG:

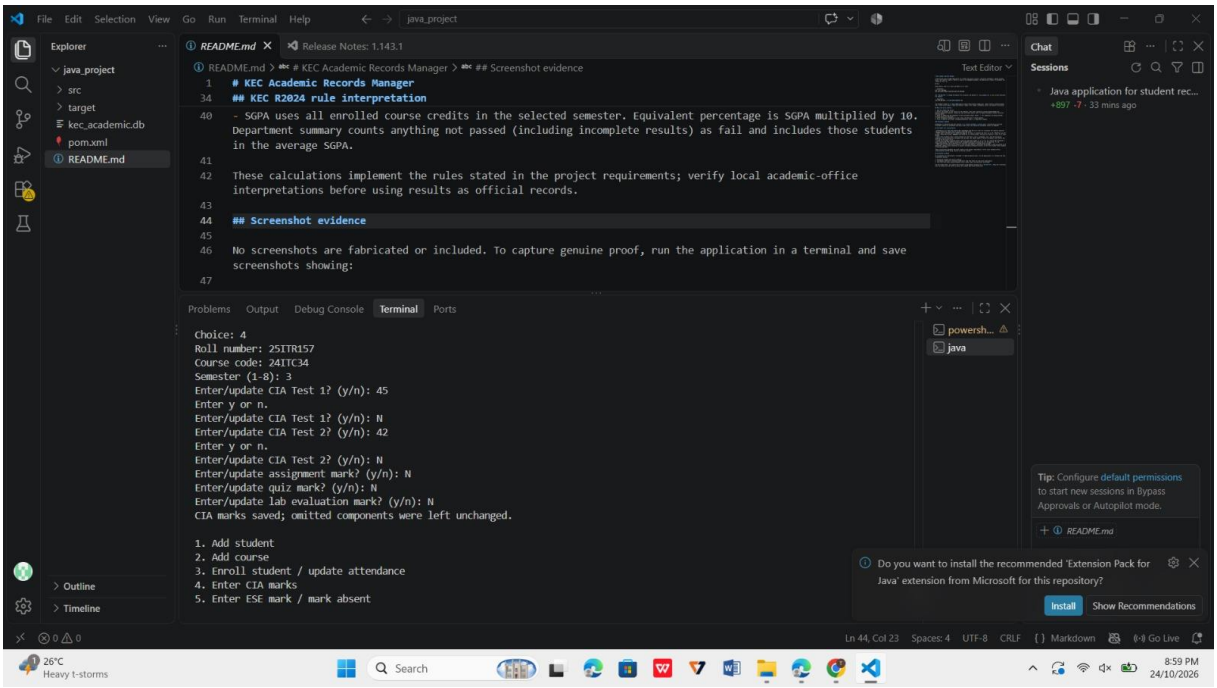


3. COURSE ENROLLMENT & ATTENDANCE TRACKING:



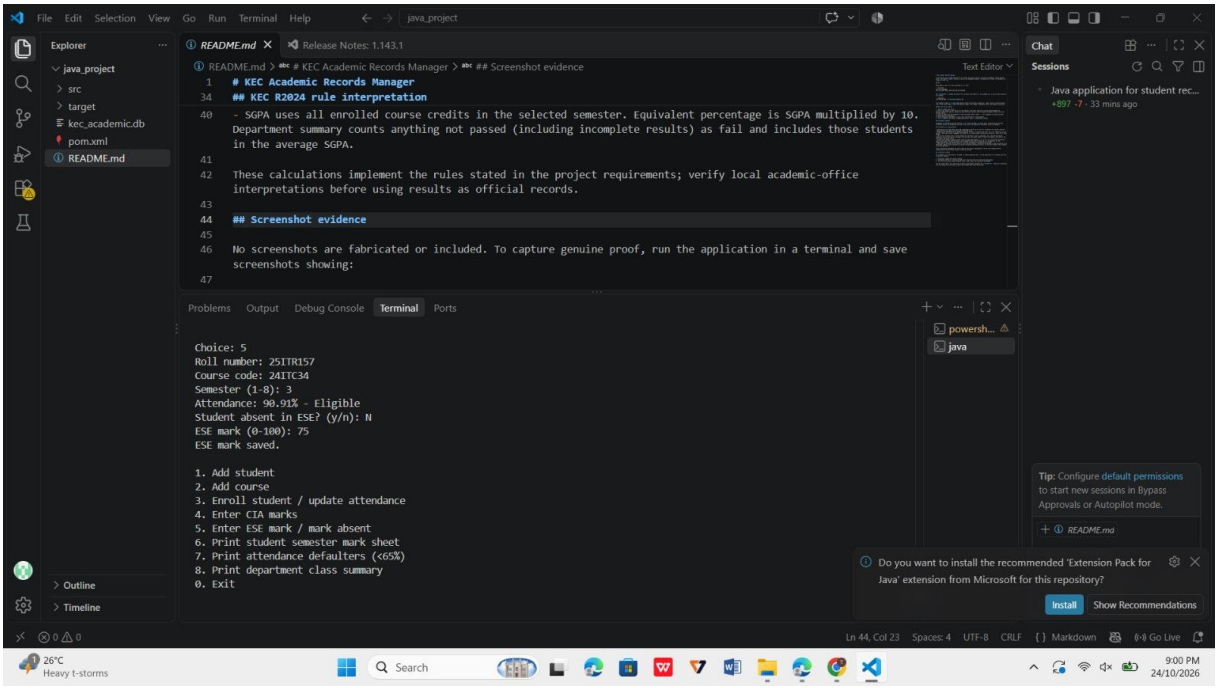
INCREMENTAL MARKS ENTRY:

1.PHASE 1 (CAT):

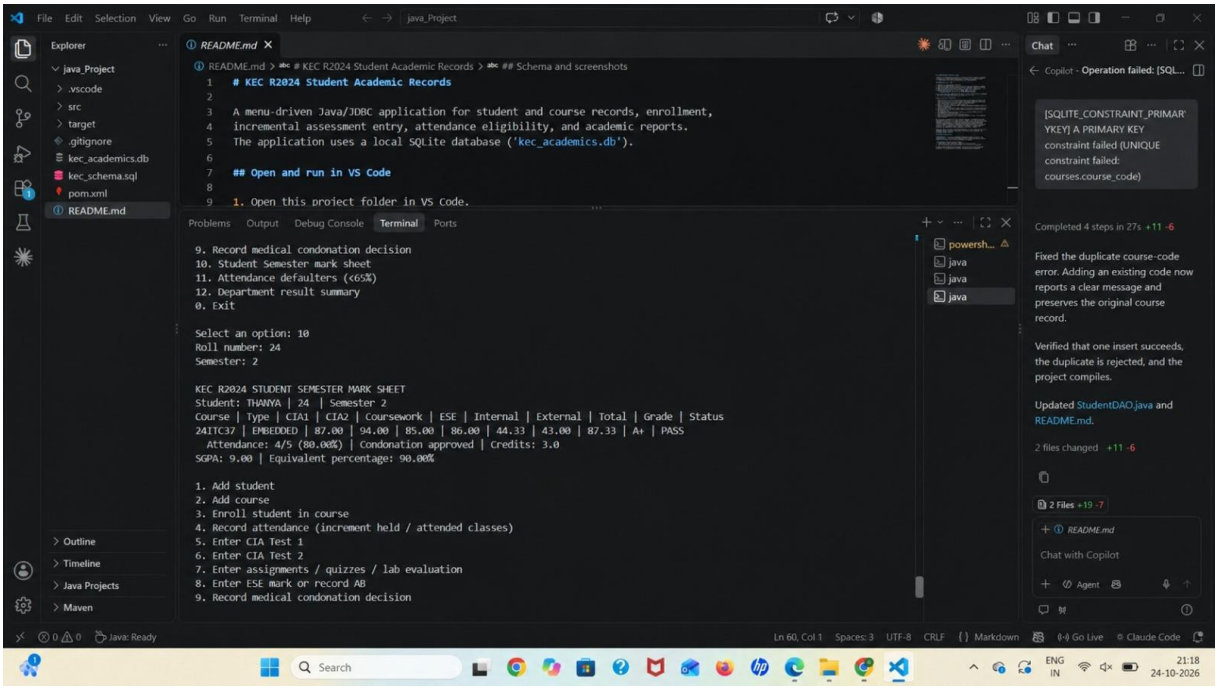


ATTENDANCE & GRADING RULES:

1.ATTENDANCE FILTER:

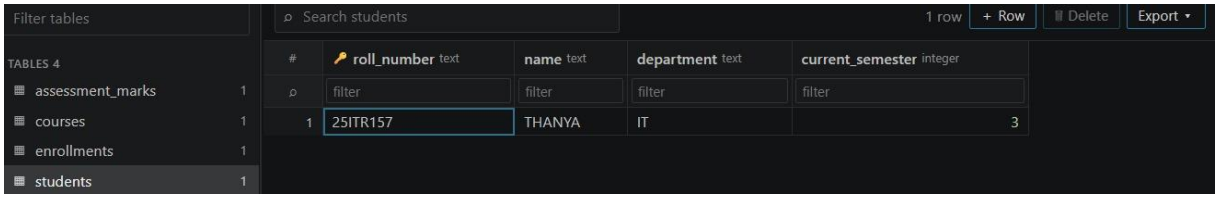


STUDENT SEMESTER MARK SHEET:

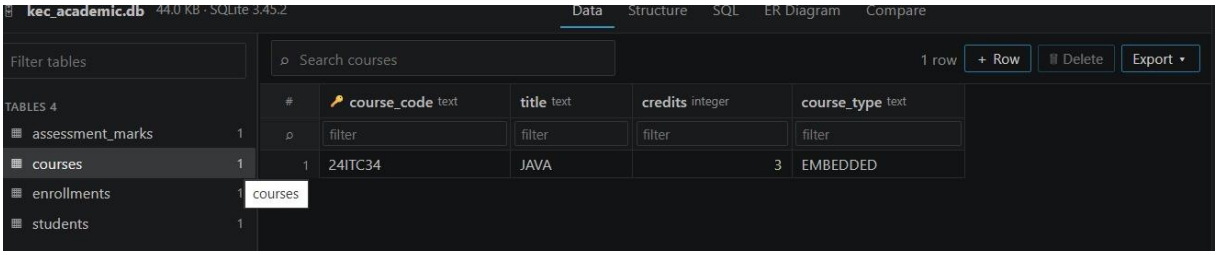


DATABASE:

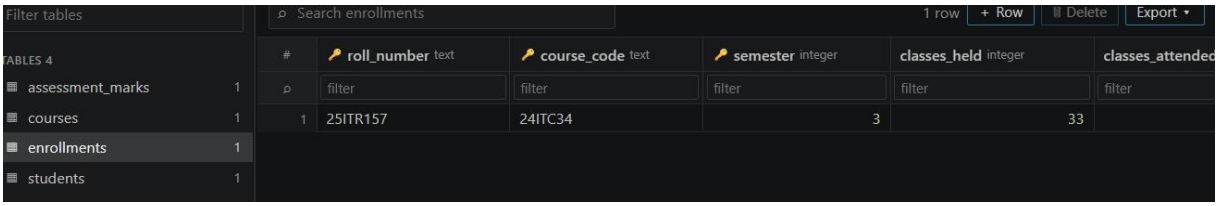
STUDENTS DATABASE:



COURSES DATABASE:



ENROLLMENTS DATABASE:



RESULT:

Thus, the Student Academic and Continuous Evaluation System for Kongu Engineering College (Autonomous) under Regulation 2024 was successfully developed and tested using Java (JDBC) and MySQL. The system efficiently handles the calculation of Continuous Internal Assessment (CIA), End Semester Examination (ESE), Letter Grades, Attendance Eligibility, and Semester Grade Point Average (SGPA).

REFERENCES:

1. **Oracle, JDBC Basics.** https://docs.oracle.com/javase/tutorial/jdbc/basics/
2. **SQLite, SQLite Documentation.** https://www.sqlite.org/docs.html
3. **SQLite, SQL Language Documentation.** https://www.sqlite.org/lang.html
4. **Oracle, JDBC Developer's Guide.** https://docs.oracle.com/en/database/oracle/oracle-database/21/jjdbc/
5. **Oracle, Java Documentation.** https://docs.oracle.com/en/java/
6. **Apache Maven, Maven Documentation.** https://maven.apache.org/guides/
7. **CodeJava, JDBC Tutorials.** https://www.codejava.net/jdbc-tutorials

GITHUB:

LINK:	https://github.com/thanya242616/java_project/tree/main/java_project